

User Manual – INI File Editor

A comment-preserving INI file editor with a graphical interface, flexible sorting, file comparison, and export to multiple formats.

Table of Contents

- 1. [Starting the Application](#)
- 2. [Interface Overview](#)
- 3. [Opening and Creating Files](#)
- 4. [Structure Overview \(Tree\)](#)
- 5. [Editing Sections and Entries](#)
- 6. [Preview and Export Formats](#)
- 7. [Sorting](#)
- 8. [Find and Replace](#)
- 9. [Comparing Files \(Diff\)](#)
- 10. [Merging Multiple Files](#)
- 11. [Undo and Redo](#)
- 12. [Switching Language](#)
- 13. [Keyboard Shortcuts](#)
- 14. [Notes on Character Encodings](#)
- 15. [Handling Duplicates \(Win32 convention\)](#)

1. Starting the Application

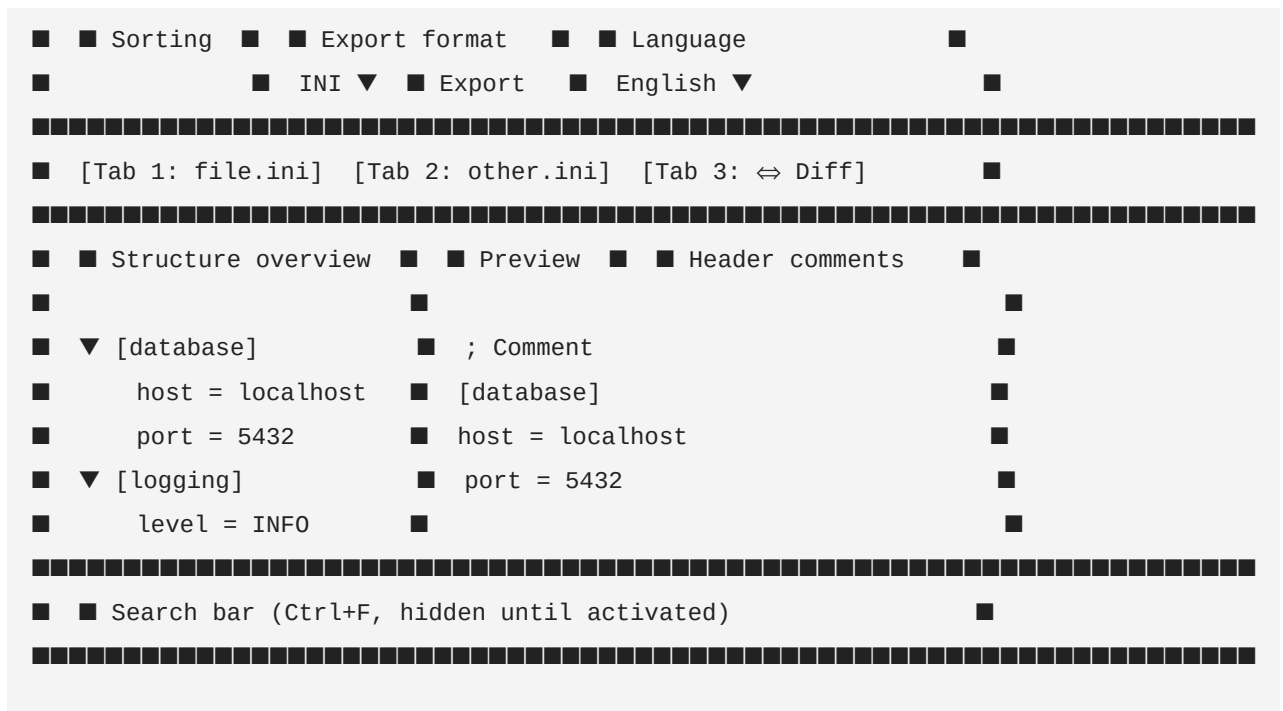
Running the executable or the project with

```
python main.py
```

On startup, the main window opens with an empty, untitled document.

2. Interface Overview





Left side – Structure overview: All sections and keys displayed as an expandable tree.

Right side – two tabs: - **Preview:** Live rendering of the file in the selected format with syntax highlighting. - **Header comments:** Global comments at the top of the file (before the first section).

Control bar at the top: Sorting, export format, and language.

3. Opening and Creating Files

Creating a new file

- Menu **File** → **New** (Ctrl+N) – replaces the content of the active tab.
- Menu **File** → **New tab** (Ctrl+T) – opens another empty tab.

Opening a file

- Menu **File** → **Open** (Ctrl+O) – opens an INI file in the current tab.
- Menu **File** → **Open in new tab** (Ctrl+Shift+O) – opens in a separate tab.
- **Drag & Drop:** Drag one or more .ini, .cfg, or .conf files directly into the window.

Character encoding: The editor automatically detects UTF-8, UTF-8 BOM, Windows ANSI (cp1252), and latin-1. No manual selection is required.

Progress indicator: When loading large files, a progress bar appears on the right side of the status bar covering both the parser and the UI build phases.

Duplicate detection: If duplicate sections or keys are found during parsing (see [Section 15](#)), the status bar shows a notice and the affected lines are highlighted in the structure view and preview.

Saving a file

- **Save** (ctrl+s) – saves the file to its original location as INI.
- **Save as** (ctrl+Shift+S) – choose a new location and file name.

Tabs with unsaved changes are marked with an * in the title. A confirmation dialog appears when closing such a tab.

Closing a tab

- Click the × on the tab header.
- Menu **File** → **Close tab** (ctrl+W).

4. Structure Overview (Tree)

The tree on the left displays the file hierarchically:

```
▼ [database]                ← Section (bold, clickable)
  host    localhost          ← Key ■ Value ■ Comment
  port    5432
▼ [logging]
  level   INFO               ; Logging level
```

Columns: Key / Section | Value | Comment | Duplicate symbol

The narrow trailing column without a header label shows the duplicate role: ★ marks the first occurrence (what Win32 actually reads, rendered in bold) and ↓ marks shadowed duplicates (rendered in italic). The whole row is also tinted (yellowish/reddish). See [Section 15](#) for details.

Navigation: - Click the arrow ▼ / ■ to expand or collapse a section. -

Expand/collapse all: Menu **View** → **Expand all** / **Collapse all**.

Synchronized scrolling (INI format only): Scrolling in the tree automatically scrolls the preview to the matching position – and vice versa.

5. Editing Sections and Entries

Editing via double-click

- **Double-click a section** → Opens the section dialog.
- **Double-click an entry** → Opens the entry dialog.

Editing via context menu

Right-clicking a section or entry opens a menu with:

Action	Description
■ Add section	Create a new section at the end
⇒■ Edit section	Change the section name and comments
■ Add key	Add a new entry to the selected section
⇒■ Edit entry	Change the key, value, and comments
■■ Delete section	Remove the section and all its entries
■■ Delete entry	Remove a single key

Entry dialog

```
Key:           [timeout           ]
Value:         [30                 ]
Inline comment: [Timeout in seconds ]
Preceding comments: [; Network setting ]
                  [; do not change ]
```

- **Inline comment:** Comment at the end of the line, e.g. `timeout = 30 ; seconds`
- **Preceding comments:** Comment lines directly above the entry (one per line)

Section dialog

```
Section name:           [database           ]
Comments (before header): [; Database connection ]
Comments (after entries): [; End database config ]
```

Editing header comments

In the **Header comments** tab on the right, global comments at the top of the file (before the first section) can be freely edited.

6. Preview and Export Formats

The **preview** on the right shows the file live in the selected format. The format is chosen via the **Export format** dropdown at the top.

Available formats

Format	Description
INI	Original format with all comments and correct structure
JSON	Compact JSON: { "section": { "key": "value" } }
XML	Valid XML with <configuration><section><entry> structure
YAML	Readable YAML (requires PyYAML)

Note: Comments are only preserved in INI format. JSON, XML, and YAML contain key-value pairs only.

Duplicate highlighting in INI format: Duplicate section headers and duplicate keys are tinted in the preview — yellowish for the first (winner) occurrence, reddish for each shadowed duplicate. JSON/XML/YAML have dictionary semantics and cannot represent duplicates, so no highlighting is applied there. See [Section 15](#).

Exporting a file

1. Select the format in the dropdown.

2. Click **Export** or use menu **File** → **Export** (Ctrl+E).
3. Choose the destination and file name.

The active sort order (if any) is applied during export.

7. Sorting

The **Sorting** dropdown at the top left controls the order in the tree and preview:

Option	Effect
No sorting	Original order from the file
Sections alphabetically	Sections from A–Z, entries in original order
Keys alphabetically	Sections in original order, keys from A–Z
Sections & keys alphabetically	Both sorted alphabetically

Sorting does **not** modify the original file. It affects the preview and export. The chosen order is written to disk only when saving.

8. Find and Replace

Find

ctrl+F opens the search bar at the bottom of the window.

■ Search text: [_____] ■ ■ Case sensitive 3 matches found ✕

- **⬅ / ➡**: previous / next match
- **Enter**: next match
- **Escape**: close the search bar
- The search operates on the **preview text** in INI format.

Find & Replace

ctrl+H additionally shows the replace row:

Green	Only in file B (added)	■
Red	Only in file A (removed)	■
Yellow	Value differs (modified)	≠
None	Identical	■

The "Duplicate (Win32 call)" column

This extra column on the right shows how Win32 applications (GetPrivateProfileString) behave with respect to duplicate sections or keys:

Symbol	Font	Meaning
★	bold	First occurrence — what Win32 actually reads
↓	italic	Shadowed duplicate — physically in the file but invisible to Win32
(empty)	normal	Unique entry, no duplicate

In the diff tab the duplicate role is intentionally rendered **without** a background tint, so it doesn't collide with the diff status colour (green/red/yellow). The comparison pairs the N-th instance from A with the N-th instance from B — a shadowed row therefore shows the *actual* value of that occurrence, not the winner value. See [Section 15](#) for details.

Options in the diff tab

Show differences only: Hides identical entries so only differences are visible.

Sorting: Independent of the source tabs, a sort order can be chosen for the comparison. This allows two files with different key orders to be compared purely by content.

Searching in the diff

Ctrl+F opens an embedded search bar directly inside the diff tab:

Search text: [_____] ■ ■ ■ Case sensitive 5 matches found (2/5) ✕

The search covers section names, key names, and values across all columns.

Live updates

The diff tab updates automatically whenever one of the source files is edited.

10. Merging Multiple Files

The merge function combines multiple INI files into a single document.

1. Menu **File** → ■ **Merge multiple files**.
2. Select one or more INI files (multiple selection supported).
3. All sections and entries are merged into the active document.

First-wins on merge: Existing keys in the target document stay unchanged — the merge function only adds missing entries. This matches the Win32 convention (see [Section 15](#)) and ensures that, when merging several configuration snapshots, the first one loaded remains authoritative.

11. Undo and Redo

Action	Shortcut
Undo	Ctrl+Z
Redo	Ctrl+Y

Every editing step (adding, deleting, editing, replacing) is saved as a snapshot. The history is discarded when the tab is closed.

12. Switching Language

The ■ **Language** dropdown in the top right switches the entire interface immediately:

- **Deutsch**
- **English**

13. Keyboard Shortcuts

Action	Windows / Linux	macOS
New	Ctrl+N	Cmd+N
New tab	Ctrl+T	Cmd+T
Open	Ctrl+O	Cmd+O
Open in new tab	Ctrl+Shift+O	Cmd+Shift+O
Save	Ctrl+S	Cmd+S
Save as	Ctrl+Shift+S	Cmd+Shift+S
Close tab	Ctrl+W	Cmd+W
Quit	Ctrl+Q	Cmd+Q
Undo	Ctrl+Z	Cmd+Z
Redo	Ctrl+Y	Cmd+Y
Find	Ctrl+F	Cmd+F
Find & Replace	Ctrl+H	Cmd+H
Close search bar	Escape	Escape

14. Notes on Character Encodings

The editor automatically detects the encoding of a file when opening it, trying the following in order:

Priority	Encoding	Typical use case
----------	----------	------------------

1	UTF-8 with BOM	Windows tools that write a BOM
2	UTF-8	Standard on Linux, macOS, modern Windows tools
3	Windows-1252 (ANSI)	Older Windows applications, German special characters
4	Latin-1	Western European systems, universal fallback

Files are always saved as **UTF-8** (without BOM). If you open an ANSI-encoded file and save it, the result will be a UTF-8 file – the content is preserved correctly.

Special characters such as ä, ö, ü, ß, and € from ANSI files are correctly recognized and displayed.

15. Handling Duplicates (Win32 convention)

In practice INI files are very often consumed by Win32 applications via `GetPrivateProfileString`. That API has a very specific behaviour for duplicate sections and duplicate keys, and the editor mirrors it exactly — what you see while editing should match what the production application actually reads.

What Win32 really sees

`GetPrivateProfileString` walks the file sequentially from the top and returns **the first match**:

1. It finds the first `[section]` with the searched-for name.
2. Inside that section, it returns the first matching key.
3. **If the key is not found there, it does *not* keep searching in later same-named sections** — the default value is returned instead.

Consequence: a second `[foo]` block with entirely different keys is **invisible** to Win32, unless those keys happen to also exist in the first block.

How the editor reflects this

Aspect	Behaviour
Parsing	Duplicate sections and duplicate keys are preserved verbatim — as separate <code>IniSection/IniEntry</code> instances in the data model. Nothing is silently merged.
Lookup	<code>IniDocument.get_section(name)</code> and <code>IniSection.get_entry(key)</code> return the first match (exactly like Win32). <code>set_entry</code> and <code>remove_entry</code> likewise operate on the first occurrence.
Round-trip	When saving, the file is reconstructed byte for byte, including all duplicates. Nothing is lost.
Detection	During parsing each duplicate is recorded in <code>IniDocument.duplicates</code> as a <code>DuplicateRecord(kind, section, key, line)</code> . The GUI uses this list to inform the user.

Visual markers in the GUI

When duplicates are detected, the editor communicates this in three places at once:

1. **Status bar** at the bottom shows "N duplicate(s) detected — affected lines are highlighted in the preview and structure view."
2. **Structure overview (tree)**: Affected rows get a tinted background and a symbol in the narrow trailing column:
3. ★ (bold) on a yellowish background — *winner*, what Win32 sees
4. ↓ (italic) on a reddish background — *shadowed*, invisible to Win32
5. **Preview** (INI format only): The corresponding lines are tinted as well — yellow for the winner, red for every shadowed duplicate.

Hovering over a marked row reveals a tooltip explaining its meaning.

Example

```

[application]
name = MyApp
session_timeout = 60           ; ← Winner: this is what Win32 sees
session_timeout = 90           ; ← Shadowed: invisible to Win32
secret_key = abc

[network]                       ; ← Winner section
host = prod-server-01

[network]                       ; ← Shadowed section: entire block invisible to Win32
failover_host = prod-server-02
timeout = 99

```

In this example: - A Win32 lookup `application/session_timeout` returns 60 (not 90). - A Win32 lookup `network/host` returns `prod-server-01`. - A Win32 lookup `network/failover_host` returns the **default value** — the second `[network]` block is completely invisible, even though it is in the file.

Behaviour in the comparison (diff)

The diff mode honours Win32 semantics in its value-comparison logic: it pairs the N-th instance from file A with the N-th instance from file B (occurrence-index pairing).

Practical consequence:

- If A has `name = MyApp` once and B has `name = MyApp` plus `name = ThomasIni`, the diff shows **two** `name` rows: the first as winner (★, both equal), the second as shadowed (↓) with `ThomasIni` on the B side.
- Shadowed values are therefore shown explicitly — they are not masked by the winner's value.

Behaviour when saving/exporting

- **INI export:** round-trip faithful, duplicates preserved.
- **JSON / XML / YAML:** these formats have dictionary semantics and cannot represent duplicates. On export the first entry wins (Win32-conformant); later entries with the same name are dropped. This is intentional — a JSON consumer should see the same value a Win32 consumer would.

When are duplicates a problem?

Duplicate entries often appear by accident, e.g. when patches are appended to a file without removing the original. They are not necessarily a bug — the editor permits them on purpose because they appear in legacy configurations and sometimes serve

as a form of documentation ("old value still visible, new value active"). If you want to clean them up, you can delete shadowed rows via the structure view's context menu.